

FDD3024: Module 3 Homework Problem Set

Roberto Guanciale and Karl Palmskog

Send your individually written problem solutions as a single PDF file by email to `smmah@kth.se`, at the latest on June 2, 2026 at 18:00. Collaboration between students and AI use is allowed, but you must write up your own solutions (handwriting allowed). Solutions are graded pass/fail based on demonstrated effort to solve the problem; a fully correct solution to all problems is not necessary.

1. Basic Hoare logic

Using the basic Hoare rules for the IMP language defined in the lecture 10 slides, derive the following partial correctness triple:

```
{1 <= N}
P := 0;
C := 1;
(while C <= N do P := P + M; C := C + 1)
{P = M * N}
```

2. Practical Hoare logic, Functional Models

Consider a function `f` on sequences/lists defined by the following equations:

```
f [] l2 = l2
f (h1::t1) l2 = h1::(f l2 t1)
```

- a) Describe briefly in words what the function computes.
- b) Formulate the function in Dafny and use it as a “functional model” for a method in Dafny that performs the same operation on arrays as the function performs on sequences. Use Dafny the method specification constructs (**requires** and **ensures**) to connect the method and your Dafny version of the function.

You do not need to verify anything using the Dafny tool, but you must argue informally why your method correctly performs the operation defined by `f` on arrays.

The following pages in the Dafny manual will be helpful:

- <https://dafny.org/latest/DafnyRef/DafnyRef#sec-sequences>
- <https://dafny.org/latest/DafnyRef/DafnyRef#sec-array-type>
- <https://dafny.org/latest/DafnyRef/DafnyRef#sec-array-to-seq>

3. Practical Hoare logic, Array Swap

Consider the following definitions in Dafny:

```
function reverse<A>(s:seq<A>) : seq<A>
{
  if |s| == 0 then
    []
  else
    reverse(s[1..]) + [s[0]]
}

method Swap<T>(a: array<T>, i: int, j: int)
  requires 0 <= i < a.Length;
  requires 0 <= j < a.Length;
  modifies a;
  ensures a[i] == old(a[j]);
  ensures a[j] == old(a[i]);
  ensures forall k :: 0 <= k < a.Length && k != i && k != j ==>
    a[k] == old(a[k]);
  ensures multiset(a[..]) == old(multiset(a[..]));
{
  var tmp := a[i];
  a[i] := a[j];
  a[j] := tmp;
}
```

We now want to implement an in-place array reversal as specified by the `reverse` function using the `Swap` method. Consider the following Dafny code skeleton:

```

method Reverse<T>(a: array<T>)
  modifies a;
  ensures a[..] == reverse(old(a[..]));
{
  var i := 0;
  while (i <= ??)
    invariant ??
  {
    Swap(a, i, ??);
    i := i + 1;
  }
}

```

Fill in the blanks (??) in this Dafny skeleton. You do not need to verify in Dafny that your completed program is correct, but you must argue informally why your invariant is correct and ensures the postcondition.